

DevOps for Digital Twins and by Digital Twins: Leveraging Virtual Replicas in Continuous Software Engineering

Gwendal Beaumont², Antoine Beugnard²,
Jannik Laval¹, Salvador Martínez², Souad Rabah Chaniour³,
Christelle Urtado³, Sylvain Vauttier³, and Bérenger Akodo¹

¹ Université Lumière Lyon 2, INSA Lyon, Université Claude Bernard Lyon 1,
Université Jean Monnet Saint-Etienne, DISP UR4570, Lyon, 69007, France

`jannik.laval@univ-lyon2.fr`

² IMT Atlantique, Lab-STICC, UMR 6285, Brest, France

`{Gwendal.Beaumont, Antoine.Beugnard, Salvador.Martinez}@imt-atlantique.fr`

³ SyCoIA, IMT Mines Ales, Ales, France

`{Souad.Rabah-Chaniour, Christelle.Urtado, Sylvain.Vauttier}@mines-ales.fr`

1 Introduction

Digital Twins (DTs) [2,4,12,15,17,20,36,39,43,51,52] are systems that fall into the category of software systems capable of interacting with their physical environment. Thus, in addition to being a faithful representation (e.g. model) of their actual system, DTs have the ability to generate their own data, autonomously. This provides them with a view of their environment, to which they can adapt and learn dynamically and continuously. DTs may also have the ability to send data and directly act on their actual system. This capability allows it to improve the efficiency of decision-making, optimization, and problem-solving. In the event of a malfunction in part of the system, it can virtually replace the dysfunctional components.

The development of DTs of systems relating to various domains has been widely explored by industry in practice and in research works [18]. An ISO norm describes the fundamentals of a framework for industrial DTs [32]. Models [21] and architectures (such as DT RAMI 4.0) exist, along with software engineering platforms (Dassault Systèmes, Siemens, ANSYS, PTC, Microsoft, etc.).

Several scientific obstacles still hinder the widespread adoption and use of DTs [18,14] among which we can highlight:

- The loss of fidelity of DTs, their recalibration and their co-evolution with the actual system [5]. The life cycle of DTs may differ from that of the actual system due to its software structure and dedicated sensors;
- The lack of tools to monitor the lifecycle of DTs [10]. Currently, DTs are designed for specific situations, making them difficult to adapt. Tools need to be developed to monitor their evolution and their relationship with the actual twin;

- The lack of human integration in the DT life cycle [22]. When the actual system evolves, the DT must adapt, as user needs also change. The DT must therefore be flexible enough to adapt to these new decision support needs on a regular basis in order to remain relevant.

Added to this, limitations in complex software systems are inherent: the complexity of the structure, which increases over time (Lehman’s law [38]), robustness and stability, cybersecurity, and energy efficiency.

All of these barriers are linked to the complex structure of a digital twin. Its various components (software, simulation, data exploitation, communication) are often addressed separately by different experts and covered in scientific literature related to industrial engineering [55].

Looking at DT from a software perspective, this chapter discusses the adoption of a DevOps culture and the use of practices and tools associated with this culture. DevOps is therefore a best practice derived from software engineering to streamline practices, automate processes and improve quality.

According to Microsoft Azure, “DevOps is the union of people, processes, and technologies designed to continuously deliver value to customers” [49]. DevOps influences the entire application lifecycle, from planning to deployment and operation, involving each role in all phases, thereby strengthening interdisciplinary collaboration within teams. This approach combines development (Dev) and operations (Ops) by promoting coordination and collaboration between traditionally siloed roles such as development, IT operations, quality engineering, and security. DevOps in software development:

- improves responsiveness to customer needs.
- increases the reliability of developed applications.
- shortens development cycles, thereby helping to achieve business objectives more quickly.

In this chapter, we show how a digital twin is a specific software in Section 2. Then, after a presentation of DevOps principles in Section 3, we discuss the integration of DevOps and Digital Twin from three perspectives: the integration of DevOps into DT design in Section 4, the integration of DT into DevOps in Section 5, a discussion about the use of DT to drive the DevOps process in Section 6.

2 Software parts of Digital Twins

Digital Twins are increasingly recognized as key enablers for the monitoring, prediction, and optimization of complex systems. While the actual entity provides observable data, the software part, commonly referred to as the digital entity, is responsible for modeling, analyzing, simulating, and influencing the behavior of the target system. This section provides an overview of the software components of DTs, with a particular focus on the digital entity and its life-cycle management.

A distinctive characteristic of digital entities lies in their continuous synchronization with physical counterparts. They collect near real-time operational and modeling data from the target entity and, in return, provide analyses, recommendations, or even control signals. In many cases, this establishes a control loop between the digital and the target entities, allowing not only observation but also direct influence over the system’s behavior.

This interaction can be fully automated or partially supervised by humans, depending on the criticality of the system and the maturity of the DT deployment, life cycle phase and use for.

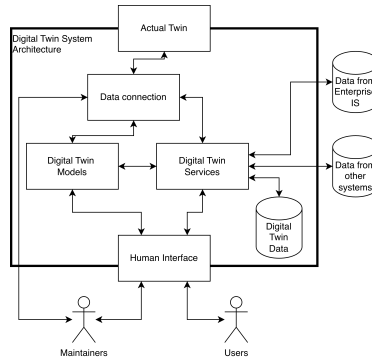


Fig. 1. Digital Twin System architecture, inspired by [45].

The life cycle of the digital entity follows a structured and iterative process, ensuring its reliability and relevance over time. According to [33], the main phases of the Digital Twin life cycle are inception, design and development, verification and validation, deployment, operation and monitoring, reassessment, and retirement. The Digital Twin life cycle is inherently iterative, with feedback loops enabling continuous refinement across phases.

The main difference between the digital entity and the target entity lies in their nature and respective life cycles. The target entity provides a functional purpose in reality and serves as the subject of the digital representation; it can be either physical or digital. Although the digital entity and the target entity share similar process groups within their life cycles, these cycles are distinct. For example, the life cycle of the target entity ends with recycling, whereas that of the digital entity concludes with retirement.

A central challenge in advancing the methodological framework of Digital Twin engineering lies in addressing the intrinsic asynchronism between the life cycles of the digital and target entities. While the digital entity interacts closely with its physical counterpart, their evolution is not necessarily synchronized. The digital entity can undergo updates, refinements, and methodological improvements independently, while still remaining aligned with the target system during operation.

The target entity provides modeling and near real-time operational data to the digital entity, while the digital entity returns analytical results to the target entity, fostering a continuous and adaptive interaction between both entities. This bidirectional exchange ensures that both systems maintain consistency and operational coherence throughout their respective life cycles.

This asynchrony becomes particularly critical when considering the availability and quality of models and the resulting Data, Information and Knowledge (DIK), or captured during the system exploitation [3]. In some cases, the target entity exists, yet the corresponding models may be outdated, incomplete, or entirely unavailable. Here, the methodology must integrate reverse-engineering strategies to reconstruct the missing or obsolete models, ensuring that the digital representation remains functional and relevant. Conversely, when the target entity does not yet exist, the approach must support the simultaneous engineering of both the system (to be) and its digital representation, maintaining coherence between the conceptual design and its eventual physical realization [25].

By explicitly accounting for these two scenarios, target entity existing but models lacking, and target entity not yet existing, the DevOps methodology can better handle the autonomy of the digital entity while ensuring operational alignment with the actual system, thereby addressing the fundamental challenge of asynchronous life cycles in Digital Twin system engineering.

3 DevOps in Software engineering

3.1 DevOps in a nutshell

DevOps, for **D**evelopment and **O**perations, refers to established software engineering practices that aim to bridge the gap between software development teams and operation teams. In other word, it aims to bridge the gap between software at design-time (when being made) and software at runtime (when being used). One of the concrete effects of DevOps practices in software engineering is speeding up software delivery and increasing software quality by automating

several steps of the software engineering process, thus systematizing them and performing them repetitively [29].

According to Humble and Farley [31], DevOps lies on four pillars: interdisciplinary collaboration, process automation, continuous monitoring and iterative improvement. The overarching goals of DevOps are to reduce organizational silos, automate repetitive tasks (i.e., tests, integration, deployment) and setup rapid feedback between development and production teams (Figure 2). These objectives are implemented through tools and practices such as continuous integration (CI), continuous deployment (CD), Infrastructure as Code (IaC) and application monitoring (AM) thus increasing application update frequency while lowering error occurrence as those processes become systematic.

3.2 DevOps phases

DevOps is generally seen as an infinite cycle that spans the whole software life. It is composed of different phases that are mostly agreed [46].

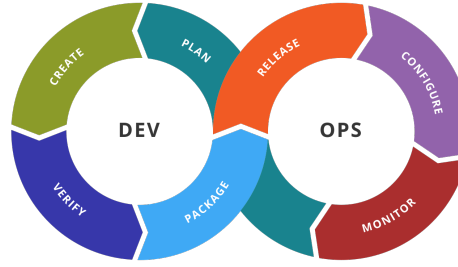


Fig. 2. A visual representation of the DevOps workflow, Kharnagy Wikimedia Commons, CC BY-SA 4.0. <https://commons.wikimedia.org/wiki/File:Devops-toolchain.svg>.

PLAN phase : During this phase, DevOps teams conduct project management tasks. They define which functionalities have to be delivered and schedule tasks to do so, using backlogs, Kanban boards and dashboards. They set objectives and acceptance criteria, prioritize tasks, synchronize dependencies and align stakeholders, risks and capabilities.

CREATE phase : This phase covers code writing, peer review and integration in short iterations. Quality and traceability are based on a branching model, systematically reviewed pull requests and unit tests run upstream. Continuous integration (CI) pipelines automate repetitive tasks to support a high pace of development without compromising the stability or readability of the source code.

VERIFY phase : This phase automatically verifies functional and non-functional requirements and continuously validates system behaviour. A pyramid of tests (unit, integration, etc.), supplemented by performance and security checks, is run in controlled ephemeral environments with controlled data sets, which promotes reproducibility and early detection of regressions. Quality thresholds determine the progress of the pipeline; any failure interrupts execution, blocks delivery and alerts the development teams.

PACKAGE phase : This phase prepares the version for deployment. The code version is pre-approved/approved, and its configuration per environment is defined (settings, secrets, migration scripts). The artefacts are grouped into a deployable bundle (binary or image, manifests), signed, versioned and stored in a registry. Delivery can be triggered via an orchestrator, with a rollback plan.

RELEASE phase : This phase releases the software application in the target environment: delivery orchestration, infrastructure provisioning and application deployment. Promotion between environments is controlled, with progressive deployments (blue/green, canary) and real-time monitoring. Rollback and restore mechanisms are ready, tested and documented to ensure controlled availability. The entire process is automated using Application Release Automation and production management tools integrated into the pipeline.

CONFIGURE phase : On the operations (Ops) side, this phase applies and maintains the post-deployment configuration: storage, databases, network, application settings and secrets. Changes are described declaratively (Infrastructure As Code) and applied idempotently via configuration automation, with drift detection and remediation. Tools such as Ansible, Puppet, and Otter ensure environment consistency and change traceability.

MONITOR phase : This phase ensures continuous visibility of the service in production in order to quickly detect any deviations. It involves collecting and correlating logs, metrics and traces, dashboards, alerts, as well as system health probes and synthetic tests. Detected incidents are handled according to run-books; lessons learned from post-incident analyses feed into the backlog and guide continuous improvement.

3.3 CI/CD Pipeline

CI/CD is an operational tool that supports the DevOps philosophy, enabling it to achieve its objectives of speed, quality and agility.

A CI/CD (Continuous Integration/Continuous Deployment) pipeline is a structured and automated set of steps that makes the software development, testing and deployment process more reliable [53,23,59]. It is one of the pillars of modern DevOps practices, promoting collaboration between development and operations teams while reducing human error and time to production.

Continuous integration (CI) involves frequently integrating code changes into a shared repository, with each integration triggering automated processes for compiling, testing, and analysing the code. This practice enables errors to be detected quickly, improves code quality, and shortens development cycles [53,59].

Continuous deployment (CD) encompasses the automation of the following steps: delivery, distribution, and deployment of software in test or production environments. CD aims to ensure that each validated change can be deployed reliably, repeatably, and quickly, up to the automatic availability of new versions to end users [53,23].

The entire CI/CD pipeline generally includes phases of compilation, unit and integration testing, security analysis, artefact creation, packaging (particularly for containerised environments), and then deployment and post-deployment monitoring. This automation increases the frequency of updates, reduces the risks associated with changes, and ensures a better user experience through faster.

3.4 DevOps and Digital Twin

The integration of digital twins into a DevOps process raises the question of how to manage the heterogeneity and complexity of these systems. Indeed, they are made up of multiple hardware and software components.

This diversity makes it difficult to accurately model the entire system and synchronise the digital twin with the reference system in real time. Another major challenge concerns the ability to detect early warning signs of hazards or failures. Using the digital twin as a proactive monitoring tool requires the implementation of advanced analysis mechanisms capable of quickly identifying abnormal behaviour or the early signs of failure.

Thus, the integration of Digital Twins and DevOps can be viewed from three perspectives (see Figure 3).

DevOps for DT : The system comprising digital and actual twins is complex and poses significant challenges [13]. It is based on models fed by data from sensors (CPS) observing the actual twin and capable of acting on it.

When discussing the development of a digital twin, we glimpse the significant complexity involved due to this architecture spanning all IT layers, from a high level of abstraction (models) to the processing of data received from sensors and the secure sending of commands, while also considering its internal functioning: simulation and decision-making support.

This leads us to consider DevOps as a relevant tool in the development of digital twins in the sense that it allows for iterative development of functionalities while ensuring that the digital twin remains operational.

DT integrated into DevOps : In this aspect, the digital twin acts as an additional control point before new features or fixes are deployed on the reference system. It can be used to improve a step or introduce an additional validation

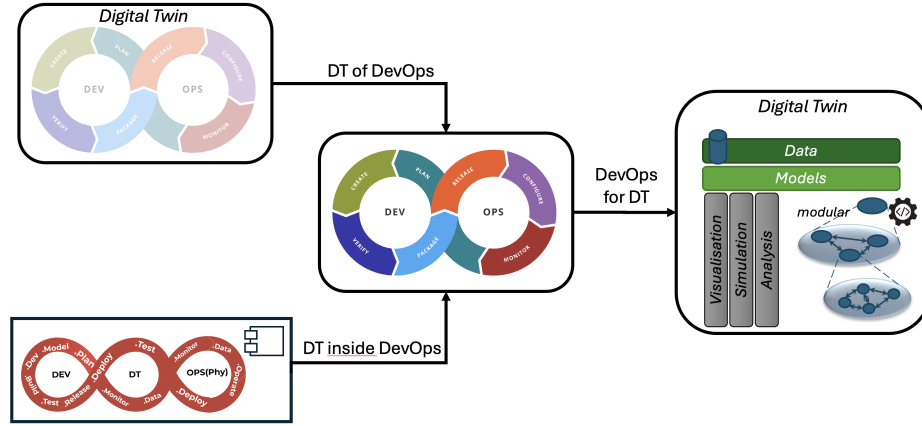


Fig. 3. Integration of DevOps and Digital Twin.

loop (DT loop). This strategy automates validation processes, both within the development cycle and in a new loop dedicated to the digital twin. By simulating the behaviour of the system in a controlled environment, it becomes possible to verify and validate the changes made, thereby reducing the risk of errors both during production and operation.

DT for DevOps : Here, the digital twin plays a role as an advanced tool for monitoring, continuous observation and detection of weak signals or hazards affecting the actual system. Thanks to its real-time simulation and analysis capabilities, the digital twin can identify anomalies or deterioration that might otherwise go unnoticed in a conventional environment.

The data collected by the digital twin continuously feeds into the DevOps loop, contributing to the development of new functional and non-functional requirements during the planning phase.

More specifically, automatic detection of a hazard systematically triggers the initiation of a new development cycle, thereby ensuring that the system dynamically adapts to actual operating conditions. This responsiveness promotes a rapid, targeted and automated response to changes in the system, while strengthening its resilience.

4 Developping Digital Twin using DevOps

The implementation of a Digital Twin relies heavily on its software component, the digital entity, which must continuously evolve alongside the actual system. As for any software system, DevOps practices play a key role in its development and maintenance by ensuring continuous integration, continuous delivery (CI/CD) but also continuous maintenance, through the monitoring of its operations (feedback loop).

Beyond the common software engineering benefits fostered by DevOps, i.e. improving the quality and reliability of the digital entity as a software, DevOps should be tailored and extended to support more specifically DTs.

4.1 DevOps: Impact on Digital Twin Life-Cycle

The main benefits of DevOps for digital entities include streamlining development and deployment processes, maintaining synchronization with the actual system during operation, and supporting rapid updates and improvements to the digital representation, thereby enhancing the robustness and adaptability of the Digital Twin:

- Version control and configuration management: Tracking changes in models, software modules, and data schemas to ensure traceability and reproducibility.
- CI/CD pipelines: Automating the build, test, and deployment process for the digital entity, ensuring rapid iterations and reliable updates.
- Automating deployment and updates: Ensuring that changes in models, software modules, or configuration can be deployed quickly without disrupting the operational twin. This can be combined with Infrastructure as Code to provision and scale automatically the resources needed for the deployments of digital twins.
- Continuous monitoring: Collecting performance metrics and operational data to detect anomalies or performance drifts.
- Feedback integration: Feeding operational insights and monitoring results back into the development pipeline to improve models and software components iteratively.
- Collaboration between teams: Facilitating interaction between software developers, data scientists, system engineers, and domain experts.

With its iterative process, including a feedback loop from operations, DevOps provides inherently a conceptual framework for monitoring and maintaining the coupling between a digital twin and its corresponding actual system.

Three main kinds of drifts are to be considered [58,1,42]:

- State (Data) drifts. A digital twin must consistently update the representation models of its actual system by periodically polling the state of its actual system or by receiving notifications of its state changes [3]. This results in maintaining a synchronization between the digital twin and the actual system during the operation phase of their life cycles thanks to the deployment of data collection and communication frameworks (sensors, software probes and monitors) (cf. Figure 4). DevOps, integrating potentially a more specialized variant like DataOps [50], can be leveraged to develop and operate these frameworks.
- Model drifts. The behavior of the actual system may evolve over time, for instance because of the wear of some physical parts or significant changes in its

usages or environment. The different models maintained by the digital twin, especially its predictive and prescriptive models [21], may then suffer discrepancies and loss of precision in monitoring and managing the behavior of the actual system. DevOps, through the continuous monitoring of the digital twin, can provide support for the early detection of those drifts and for triggering the development of updated versions of the models. If these predictive and prescriptive models are based on machine learning, MLOps [37,57,42], the variant of DevOps managing development and operation pipelines for ML models can be used. MLOps provide schemes for monitoring the data used by the digital twin, allowing drift detections thanks to live statistical analysis. Moreover, MLOps supports the continuous training of ML models, as a means to continuously adapt the predictive and prescriptive models of the digital twin to the observed evolutions in the behavior or usages of the actual system.

- Architectural drifts. actual systems may also undergo changes in their architectures such as the addition, deletion and replacement of components or subsystems. As digital replicas, digital twins must evolve to remain faithful representations of their actual systems. Depending on the precision and fidelity of its representations, digital twins may also become substitutes for their actual systems in different phases of their life cycle (design, test and even operation) [16]. This means that digital twins may exist before their actual systems and be used as blueprints for their development [27]. This advocates for digital twins that mirror not only the states and behaviors of actual systems but also their architectures.

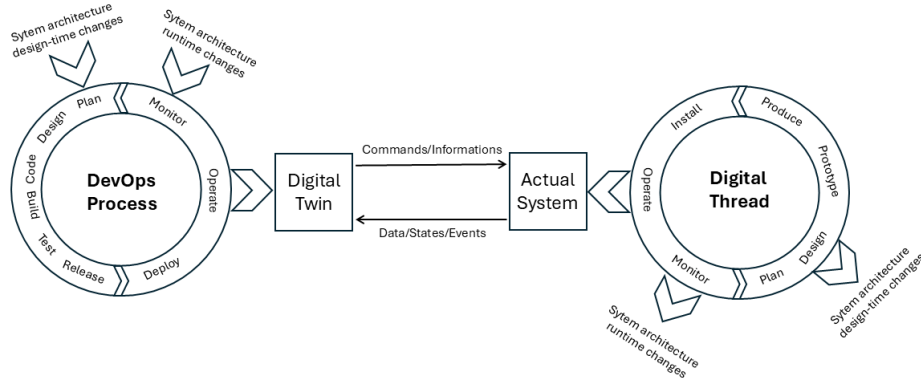


Fig. 4. Synchronizing the life cycles of a DT and its corresponding actual system [9].

DevOps infrastructures facilitate the management of complex, modular software architectures (version control and configuration management, automated build and test, automated deployment) and provides thus inherently support for

the development, deployment and maintenance of Digital Twins. Besides, microservices are considered a privileged paradigm for implementing digital twins [56]. Indeed, the data-centric, siloed but composable philosophy of microservices suits the concept of digital twins, as they are intended to "progressively aggregate data in order to represent different facets of the same object, thereby creating a holistic representation accessible to several applications" [16]. Research states that DevOps and microservices architectures have mutual benefices [8,60]. This combination currently corresponds to the best practices recommended by the SOTA in software engineering.

Current research on DevOps studies also the application of model-based software engineering concepts, to facilitate DevOps thanks to specialized/ pivot modeling languages and generative approaches [13]. Model-based DevOps would produce models for the design, development and deployment of the DTs all along their life cycles. Architecture models produced by model-driven DevOps practices would help detect and manage architectural drifts.

Analogous to DevOps, the Digital Thread concept corresponds to a complete digitalized management for the lifecycle of products [44,26]. The digital thread of the actual system will track all the architectural changes made during the development, deployment and operation phases of its lifecycle. These changes can be notified to the DevOps framework managing the Digital Twin in order to trigger architectural evolutions and fix architectural drifts (cf. Figure 4) .

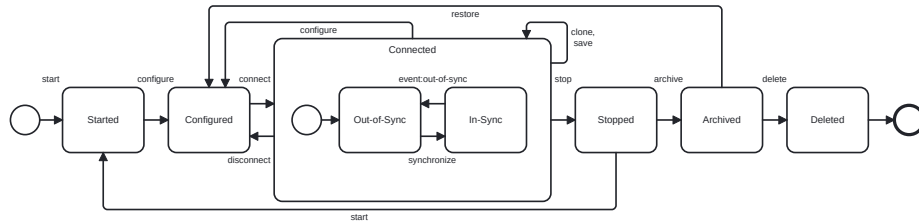


Fig. 5. Simple DT life cycle as a state-machine [9].

4.2 Adapting DevOps to DT management

Approaches to integrate DevOps practices into the life cycle management of DTs have emerged. To achieve such goal, DT life cycle operations are formalized and mappings to DevOps elements are proposed to create actionable software development and management frameworks [48,11].

Insuring the operationalization of DTs is a crucial step towards providing accurate and up-to-date information to users. Issues in this process, removes the benefits of using a DT. Thus, a flawless continuity of the DT life cycle contributes to better quality of services [48,40]. Refinement and formalization of the DT operationalization phase in the DevOps process exhibits many operations

[48,46]. These operations fall into two categories [9]: i) operations related to the actual system, e.g., the replacement of a physical part on a physical system, ii) operations related to the DT as a software application, e.g., the deployment of the DT to a platform. Operations of type ii) are further refined into two sub-categories: tool-specific operations and generic operations. Figure 5 summarizes the generic ‘core’ life cycle for the operation of DTs, as proposed in [9].

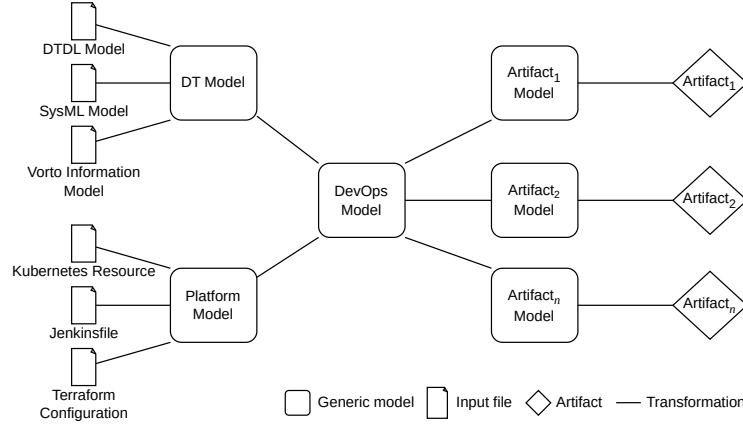


Fig. 6. Generic DevOps artifacts generation from DT models and a Platform model [9].

In this state-machine diagram, each life cycle operation appears as a transition and each target state is named operationed. The only exceptions are the In-Sync and Out-of-Sync states which are inside the Connected state. This is because the DT can transition from being In-Sync or Out-of-Sync when the connection is established to its associated system [48]. While the synchronize operation allows to enter the In-Sync state, the transition to Out-of-Sync happens through events such as an evolution of the system or a drift notified to the DT.

Operationalization of DTs leveraging DevOps also implies redesigned workflows to be integrated to DevOps frameworks. Figure 6 depicts how DevOps supports DT life cycle management. At the beginning of the process, input files from different technologies feed two kinds of models: a) a DT model and, b) a Platform model. The former is a DT model, defined in a pivot DT modeling language, which aggregates information from different sources to provide a comprehensive, standardized overview of the DT. The latter is in the same way a platform model, defined with a pivot platform modeling language, that also aggregate the information defining the target platform on which the DT will be installed. These models are used to instantiate a tailored DevOps process, using a mapping of the generic life cycle operations for DTs with a pivot DevOps process definition language, that will be used to generate models of the

different artifacts needed to deploy the DT on the target platform. Finally, using Model-to-Text transformations, generated artifacts can be installed on the target platform to provide DT-related operations.

Figure 7 shows a concrete implementation of this conceptual DevOps framework for DTs, combining popular industrial tools: DTDL [34], as a DT modeling language and GitOps [61], as a deployment and operation manager.

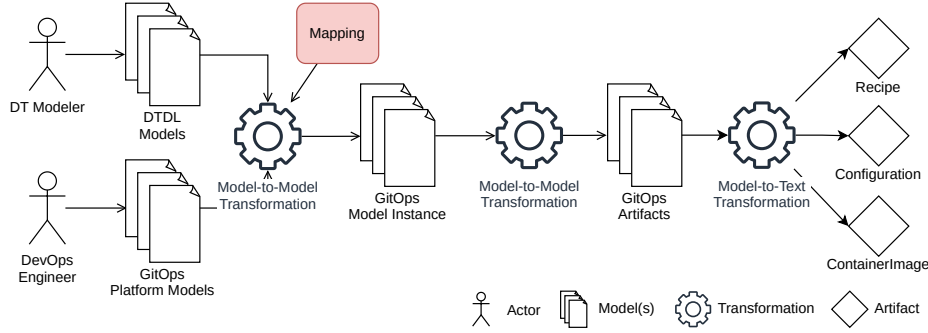


Fig. 7. DevOps artifacts generation from DTDL models and a GitOps Platform model [9].

4.3 Contributions of Digital Thread to DevOps for the Digital Twin

The Digital Thread is a cornerstone in the implementation of agile and automated approaches such as DevOps applied to Digital Twins. By ensuring the continuity, traceability and integration of data and models throughout a system's lifecycle, it provides a robust foundation for continuous development and operational pipelines. Unlike traditional approaches, which are often limited to specific exchange formats or interfaces, the Digital Thread systemics interoperability and reduces dependency on ad hoc integration code, which is difficult to maintain [26]. In the case of the Digital Twins, for example, it provides consolidated, validated data to feed simulation, analysis and optimisation environments [62]. In practice, this translates into greater reliability of the information injected into the Digital Twin, which reinforces the relevance of the decisions resulting from the DevOps processes. In fact, Digital Thread and DevOps are two distinct but intrinsically linked concepts. From one side, the Digital Thread represents a holistic vision of a continuum of data and models, connecting the different phases of a product or system's lifecycle [26]. From the other side, DevOps represents a set of practices and tools that aims to accelerate, automate and improve the reliability of software development and its operational deployment. In the context of Cyber-Physical systems (CPS), this complementarity is reinforced by the emergence of ModDevOps, an extension of DevOps that integrates Model-Based Engineering (MBE) as well as advanced verification and validation ca-

pabilities. ModDevOps treats models as first-class abstractions, which resonates directly with the Digital Thread philosophy. In addition, the automation offered by CI/CD pipelines enables data and models to be continuously transformed, integrated and updated, ensuring the consistency and sustainability of the Digital Thread [29].

Within the DT context, the relation between Digital Thread and DevOps plays a crucial role in ensuring effective support and deployment of DTs. As dynamic virtual representations of physical assets or systems, DTs require a constant flow of reliable, up-to-date information to maintain their operational value. Digital Thread provides this informational backbone by consolidating and contextualizing data from multiple disciplines [62]. DevOps, and more specifically ModDevOps, ensures the implementation of an agile and automated process to maintain this flow, adapt it to changes and exploit it in complex and heterogeneous environments [30]. This synergy not only accelerates the development and deployment of Digital Twins, but also enhances their ability to support real-time decision-making, diagnosis and optimization.

4.4 Related Works

The literature shows a growing interests in modeling Digital Twins [10,51,52,43,40]. In our context, the DevOps application to Cyber-Physical Systems dates appeared even before its application to DTs [39,15,20]. More recently, work from Koren et al.[36] show-cased the DevOps4CPPS framework, spotlighting the use of microservices and GitOps in a CPS environment.

Back to the application of DevOps to DTs, Aissat et al.[2] introduce JuNoOPS, a DevOps framework establishing guidelines on using DevOps for DTs. Based on their experiment, they highlight the difficulties of collaboration between domain and non-domain experts (e.g. git training). Other works investigate the use of DevOps to manage the lifecycle of DTs through the generation of DevOps artifacts [9].

More technical works imply crafting DTs platforms centered around DevOps technical tools, such as in Talasila et al.[56] and Dalibor et al. [17]. Their platforms allow users to install, deploy, and ultimately share DTs between them. Concurrently, Colantoni et al. [12] propose DevOps-ML, a framework that allows users to model DevOps platforms and engineering processes to be used in the context of DTs.

5 Integrating Digital Twin in the DevOps process

5.1 Managing operations with DTs of Software Systems

In a traditional DevOps practice, it is natural to bring DEV and OPS practices closer together. However, it focuses on team collaboration to stay as close as possible to the need, and less on the development process. For example, testing and validation prior to deployment is carried out offline, e.g. disconnected from

the actual system. Therefore, if the test data is not up to date, the deployment could go wrong and require a rollback procedure.

To address this issue, the community has developed strategies, including DevTestOps, by integrating continuous testing into the DevOps loop [6]. Digital Twin can therefore be used to provide a dynamic virtual representation of the system under development [41,42].

If we integrate a Digital Twin into the DevOps loop, then developers' testing capabilities are increased tenfold in the sense that the data is up to date, the risk of bugs is limited by the simulation provided by the Digital Twin, and the possibility of improvement is supported by the almost immediate feedback from the digital twin.

Digital Twin makes it possible to deploy software in an almost real environment, with data and expected behaviour close to that of the reference system at the same moment. It offers a more interesting level of environmental complexity than a simple software testing environment.

In the rest of this section, we discuss the integration of the Digital Twin in two different ways. The first is the integration of the DT into each stage of DevOps (Section 5.3), the second is the addition of a loop in order to integrate the DT as a stage of DevTwinOps (Section 5.4). Before that, we present some ideas for integrating the Digital Twin into the DevOps phases (Section 5.2).

5.2 In which phases can DTs be useful?

This subsection details the list of DevOps steps and provide ideas on how each step can be improved by a digital twin, as shown in Figure 2. In the Plan requirements and Code phases, DT can provide a sandbox for the development team to understand, test and visualise the needs of the operational teams. In the Build phase, DT can be used to simulate different configurations and scenarios. Bugs can be identified and corrected before deployment phase. In the Test phase, a digital twin allows one to create a simulation environment that is close to the actual system, facilitating testing without affecting the infrastructure. Tests can be automated and run continuously to quickly detect potential problems without affecting the real system. For example, by simulating different attacks or security breaches in a DT, teams can identify and fix vulnerabilities before they are exploited. In the Release phase, a digital twin can maintain an accurate representation of the current state of the system, thereby facilitating configuration management and ensuring that changes are audited. In the Deploy phase, DT can be used to automate the deployment process by simulating potential impacts before applying changes to the actual infrastructure. In the Operate phase, the DT can be used to analyse and optimise system performance by simulating different configurations and load increases. In the Monitor phase, the Digital twin can integrate real-time monitoring data, enabling teams to monitor system status and quickly identify anomalies. By combining predictive models with actual data, the DT can predict how the system will respond to different future scenarios and thus help diagnose problems before they become critical.

5.3 TwinOps: testing performed by a DT in the DevOps process

TwinOps, as presented by Hugues et al. [29], offers an integrated approach that combines model-based systems engineering (MBSE), digital twins, and DevOps practices for Cyber-Physical systems (CPS) engineering. This unified process aims to leverage models as central artefacts, from requirements definition to code generation, simulation, deployment, and operational data collection.

At the heart of TwinOps lies the use of models as first-class artefacts for analysing, simulating, and generating parts of the system. TwinOps focuses primarily on industrial or cloud environments, requiring complex infrastructure for continuous data synchronisation. The TwinOps modelling phase encompasses requirements definition, architecture and component modelling, and virtual model integration to assess compliance with metrics. The Test Bench/System Realisation phase acts as an automated software factory that builds various artefacts, including code for simulation and executables for the target, while also producing glue code for monitoring and detecting specific execution patterns. The OPS phase deploys the generated software, collects data and analyses it, based on indicators generated from the models.

More specifically, TwinOps is composed by the following phases:

- Plan requirements: This phase frames and prioritises functional and non-functional requirements (latency, safety, traceability), Cyber-Physical integration constraints and acceptance criteria, making them measurable from the outset through metrics and validation thresholds. It feeds into a fully traceable backlog and prepares the continuous improvement loop [28].
- Modeling architecture: it formalises the architecture and its components (e.g. SysML for requirements and interfaces, AADL for software/real-time architecture, Modelica for the physical model) in an encapsulated modelling environment (containers) to ensure the reproducibility of tools and versions. It produces interface contracts, assumptions and a model integrating the physical and software description that can be used subsequently.
- Virtual integration: it aims to detect integration incompatibilities early on by running interaction scenarios between components on models. It relies on scriptable tools integrated into a CI/CD chain to orchestrate the behaviour of the DT and execute automated verification plans.
- Code generation: it automatically (or semi-automatically) produces the application and middleware code from the models and interface contracts, as well as the simulation artefacts. It generates the instrumentation glue code to track resources and data exchanges and detect execution patterns. The outputs are versioned, reproducible and integrated into the CI build.
- Testbench assembly: it assembles the components into execution targets: the software intended to run on the target platform and the variable fidelity system simulators (software in the Loop/Hardware in the Loop) in order to validate behaviours prior to deployment.
- Run / Simulate: it first validates the scenarios in closed-loop simulation on the DT and on an instrumented bench, then on the actual system. Deploy-

- ment is carried out in stages (staging: development – integration – production) with output control at each stage. This controlled progression reduces operational risk while maintaining the delivery pace.
- Monitor: it establishes end-to-end observability: metrics, real-time logs and traces, event correlation, resource monitoring, and detection of data and model drift. Monitoring feeds the development backlog, secures operations through actionable alerts, and ensures traceability/auditability.
 - Data analysis: it compares the system’s observation results with the DT’s predictions: calculation of deviations/residuals, explanation of performance and identification of probable causes. Detects deviations (data, sensors, models) and proposes actions (recalibration, retraining, parameter/model updates). Produces automated reports and KPIs that close the TwinOps loop and justify feeding the backlog.

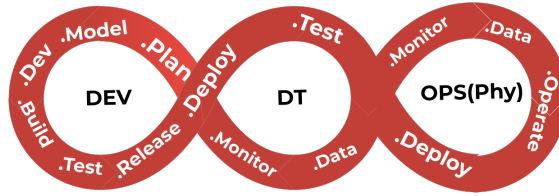


Fig. 8. DevTwinOps Methodology.

5.4 DevTwinOps : how to integrate a DT in a DevOps process

Work on integrating digital twins into DevOps processes has highlighted solutions to various operational needs, facilitating both the implementation and acceleration of DevOps practices. DT contributes to the optimisation of integration and continuous delivery cycles, while enhancing the reliability and quality of deployments on the reference system.

DTOps [47] proposes a first idea of how to integrate a DT in a DevOps process. In the work presented in Figure 8, we aim to propose a complementary methodology by exploiting the full capabilities of the DT, deploying the source code on a DT before going into production, and using this DT to monitor the reference system. Thus, the process consists of three modular loops:

The **Development Cycle (DEV)** aims to create, model and prepare software artefacts integrating the new functionality of the reference system, based on a model-oriented approach. The development cycle comprises six key stages: plan, model, develop, build, test, release.

- Plan: This initial phase consists of defining the project (or feature) objectives, functional and non-functional requirements, and the scope of development. In the context of digital twins, planning also includes defining the abstractions and commands to be modelled, the scenarios to be simulated, and the indicators to be monitored via the digital twin.
- Model: This phase involves developing abstract models that describe the expected behaviour of the system, both in terms of software components and interactions with hardware and sensors. These models form the basis of the digital twin and enable the behaviour of the system to be simulated, analysed and anticipated in various scenarios. This approach facilitates automatic code generation, consistency between design and implementation, and continuous integration of changes.
- Dev: This stage involves creating, modifying and managing source code (applications, middleware, IoT communication scripts, etc.), based on the models established in the previous phase. Manual implementation of parts not covered by automatic generation (e.g., specific algorithms, advanced data processing, high-level logic). And finally, implementation of features, bug fixes, and preparation of the software for the next phases.
- Build: The code is compiled and assembled into executable artifacts. This step is automated in DevOps pipelines via continuous integration (CI) tools, ensuring that the software is always in a deployable state.
- Test: Automated tests are run to verify functional compliance (unit, integration, and anomaly scenarios), performance, security, and robustness of the software.
- Release: Once validated, the software is prepared for deployment on the digital twin. This step includes version management, documentation, and coordination with operational teams.

The **Digital Twin Cycle (DT)** aims to validate new features. Based on the DT's features, this cycle makes it possible to detect and correct any design or implementation errors in a controlled environment using current data coming from the actual system, thereby reducing the risk of incidents in production. The DT loop comprises four key stages: Deploy, Test, Data, Monitoring.

- Deploy: The first step is to install the version of the code validated in the Build stage on the digital model of the system (the digital twin). Deployment on the digital twin allows the impacts of the changes to be assessed.
- Test: after deployment, test scenarios are run in the simulated environment. These tests cover a variety of situations, ranging from normal conditions to extreme cases, in order to verify that the functionality responds as expected and to identify any undesirable or unexpected behaviour. Functional scenarios need to define normal use cases (obstacle detection, sensor data acquisition, parameter regulation). Then, contingency scenarios define extreme situations (network packet loss, CPU overload, sudden variation in sensor measurements) to assess robustness. The tests must use MIL/SIL/HIL methods ::MIL (Model-in-the-Loop) to verify the functional logic of the

feature model, SIL (software-in-the-Loop) to ensure that the generated code behaves as expected, HIL (Hardware-in-the-Loop) to test hardware integration before actual deployment.

- Data: On the digital twin, all relevant data (performance metrics, event logs, system statuses) is collected in real time. This step ensures complete traceability of results and forms the basis for in-depth functionality analysis.
- Monitoring: The collected data is then analysed. The results are compared with the initial expectations defined in the Plan stage, enabling discrepancies to be identified, the root causes of any anomalies to be understood, and essential feedback to be provided to guide subsequent iterations and improve the model or algorithm.

The **Actual System Cycle (OPS)** aims to deploy the functionalities previously validated in the virtual environment of the digital twin onto the physical system, then to operate them in real conditions. The OPS loop comprises four key stages: Deploy, Operate, Data, Monitoring.

- Deploy: This first step involves updating the reference system with the validated version of the functionality (code, configuration, settings) once testing has been successfully completed in the digital twin. This operation can be automated via a CI/CD pipeline, including update scripts and post-deployment checks (version hash, file integrity, hardware compatibility).
- Operate: Once the new feature has been deployed, it is integrated into the daily use of the reference system. This phase involves using the physical system in an operational context while ensuring that the feature meets the performance, security and reliability requirements defined during the DEV loop planning phase.
- Data: During operation, operational data is collected continuously: performance metrics, event logs, system status, etc. This systematic collection allows the actual behaviour of the system to be documented and monitoring to be prepared.
- Monitoring: The data collected is then analysed during this monitoring phase, with particular attention paid to detecting any discrepancies compared to the results obtained in the simulation. This feedback is essential for identifying any discrepancies and anticipating incidents. The DT is particularly useful here for the early identification of weak signals leading to a failure.

In this approach, the digital twin acts as an additional control point before new features or fixes are deployed to the reference system. It is introduced as an additional validation loop (DT loop). This strategy automates validation processes, both within the development cycle (DEV loop) and in a new loop dedicated to the digital twin. By simulating the behaviour of the system in a controlled environment, it becomes possible to verify and validate the changes made, thereby reducing the risk of errors both during production and operation.

As a result, this approach helps to optimise integration and continuous delivery cycles, while enhancing the reliability and quality of deployments on the

reference system. However, it can also be used as an OPS loop supervisor. With monitoring and control of deviations, it can trigger maintenance operations (security updates, manual maintenance requests, etc.).

This section has outlined a number of proposed and developing approaches to integrating DT into DevOps at various levels: from integration into a single process stage to integration into an entire process phase. We can see the benefits of DT for DevOps, improving the development process and the quality of the applications developed.

6 Conclusion: Towards a reflexive, unified Digital Twin DevOps

We believe that digital twins can be used for intangible entities such as processes or software applications. The premise is that if we can visualize them, we can create their digital twin. We are now seeing digital twins of manufacturing processes, supply chains, financial products, and other intangible entities [27].

The principle of continuous improvement advocated by DevOps for developed software tools helps to improve software development. That said, the DevOps process itself must be scalable and adaptable to its environment. Developers must keep it alive over time, monitor its indicators and improve it. To do this, we could rely on a DT of the DevOps process.

Using DT in conjunction with the DevOps process would improve the process as well as the product. If we manage to use DT for DevOps, and use DevOps for DT design, then it may be possible to manage the entire DevOps Pipeline by feeding back information from DT. In this way, DT could optimise software development processes as a whole [35] and thus also improve itself, as is done on CPS systems [19] with self-adaptation.

In this case, we enter into a self-adaptive system containing hardware (CPS), software, combined with process management.

The MAPE-K model (Monitoring, Analysis, Planning, Execution over a shared Knowledge base) [24] provides a fundamental framework for the design of autonomous and self-adaptive systems. This model, introduced by IBM in the context of autonomic computing, structures adaptive behaviour around a feedback loop consisting of four main phases: monitoring the state of the system, analysing the collected data, planning adaptive actions, and executing these actions, all based on a shared knowledge base, shown in Figure 9.

In 2015, Arcaini et al. [7] specified that MAPE-K allows for the formal modelling of self-adaptation processes, clearly separating the responsibilities of each phase and facilitating the verification and validation of adaptive behaviour. According to [54], the integration of a central knowledge base into the MAPE-K loop promotes effective information management, which is essential for decision-making in dynamic and uncertain environments.

Thus, the MAPE-K loop is now widely recognised as a pillar of self-adaptive system architectures, particularly in the fields of cloud computing, IoT and Cyber-Physical systems, where adaptability and resilience are key requirements.

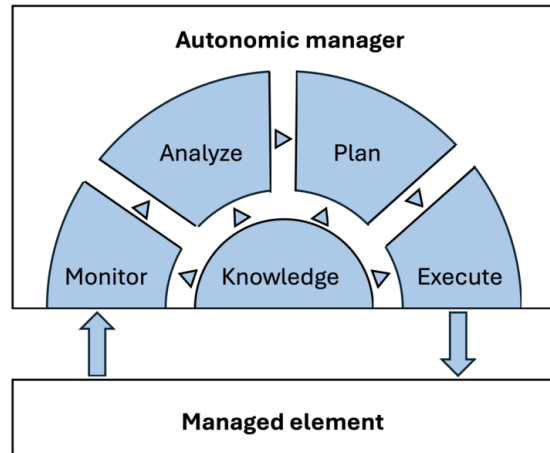


Fig. 9. MAPE-K principles [24].

To date, the use of the MAPE-K loop to develop a digital twin of the DevOps process is not offered, but research is progressing.

References

1. Abbasi, F., Brimont, P., Pruski, C., Sottet, J.S.: Understanding semantic drift in model driven digital twins. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. p. 419–430. MODELS Companion '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3652620.3688256>, <https://doi.org/10.1145/3652620.3688256>
2. Aissat, S., Beaulieu, J., Bordeleau, F., Gascon-Samson, J., Poirier, E., Motamedi, A.: JuNo-OPS: A DevOps Framework for the Engineering of Digital Twins for Built Assets. In: Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. pp. 496–506. MODELS Companion '24, Association for Computing Machinery, Linz, Austria (2024). <https://doi.org/10.1145/3652620.3688266>
3. Alaoui, M.E., Rabah, S., Chapurlat, V., Richet, V., Plana, R.: An original data, information and knowledge management approach for model-based engineering projects. IFAC-PapersOnLine **55**(10), 3262–3267 (2022). <https://doi.org/https://doi.org/10.1016/j.ifacol.2022.10.135>, <https://www.sciencedirect.com/science/article/pii/S2405896322021486>, 10th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2022
4. Alghamdi, W., Albassam, E.: Synchronization patterns for digital twin systems. Journal of Applied Data Sciences **5**(3), 1026–1037 (2024). <https://doi.org/10.47738/jads.v5i3.267>, <https://www.bright-journal.org/Journal/index.php/JADS/article/view/267>
5. Ali, Z., Biglari, R., Denil, J., Mertens, J., Poursoltan, M., Traoré, M.K.: From modeling and simulation to digital twin: evolution or revolution? SIMULATION

- 100(7), 751–769 (2024). <https://doi.org/10.1177/00375497241234680>, <https://doi.org/10.1177/00375497241234680>
6. Angara, J., Gutta, S., Prasad, S.: Devops with continuous testing architecture and its metrics model. In: Sa, P.K., Bakshi, S., Hatzilygeroudis, I.K., Sahoo, M.N. (eds.) *Recent Findings in Intelligent Computing Techniques*. pp. 271–281. Springer Singapore, Singapore (2018)
 7. Arcaini, P., Riccobene, E., Scandurra, P.: Modeling and analyzing mape-k feedback loops for self-adaptation. In: *IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. pp. 13–23 (2015). <https://doi.org/10.1109/SEAMS.2015.10>
 8. Balalaie, A., Heydarnoori, A., Jamshidi, P.: Microservices architecture enables devops: Migration to a cloud-native architecture. *IEEE Software* **33**(3), 42–52 (2016). <https://doi.org/10.1109/MS.2016.64>
 9. Beaumont, G., Beugnard, A., Martínez, S., Urtado, C., Vauttier, S.: Towards automating the life cycle management of digital twins. In: Bork, D., Lukyanenko, R., Sadiq, S., Bellatreche, L., Pastor, O. (eds.) *Conceptual Modeling*. pp. 412–430. Springer Nature Switzerland, Cham (2026). https://doi.org/10.1007/978-3-032-08623-5_22
 10. Bordeleau, F., Combemale, B., Eramo, R., van den Brand, M., Wimmer, M.: Towards model-driven digital twin engineering: Current opportunities and future challenges. In: Babur, Ö., Denil, J., Vogel-Heuser, B. (eds.) *Systems Modelling and Management*. pp. 43–54. Springer International Publishing, Cham (2020)
 11. Béchu, G., Beugnard, A., Cao, C.G.L., Perez, Q., Urtado, C., Vauttier, S.: A software engineering point of view on digital twin architecture. In: *IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA 2022)*. pp. 1–4 (Sep 2022). <https://doi.org/10.1109/ETFA52439.2022.9921617>
 12. Colantoni, A., Berardinelli, L., Wimmer, M.: DevOpsML: towards modeling DevOps processes and platforms. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. pp. 1–10. *MODELS '20*, Association for Computing Machinery, New York, NY, USA (Oct 2020). <https://doi.org/10.1145/3417990.3420203>
 13. Combemale, B., Jézéquel, J.M., Perez, Q., Vojtisek, D., Jansen, N., Michael, J., Rademacher, F., Rumpe, B., Wortmann, A., Zhang, J.: Model-based DevOps: foundations and challenges. In: *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. pp. 429–433. IEEE (2023)
 14. Combemale, B., Vicat-Blanc, P., Blouin, A., Bril El Haouzi, H., Bruel, J.M., Deantoni, J., Duval, T., Gérard, S., Jézéquel, J.M.: Engineering Digital Twins: A Research Roadmap. In: *Proceedings of the 2nd International Conference on Engineering Digital Twins (EDTconf 2025)*. pp. 1–7. Grand Rapids, Michigan, United States (Oct 2025), <https://inria.hal.science/hal-05223776>
 15. Combemale, B., Wimmer, M.: Towards a Model-Based DevOps for Cyber-Physical Systems. In: Bruel, J.M., Mazzara, M., Meyer, B. (eds.) *Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment*. pp. 84–94. Springer International Publishing (2020). https://doi.org/10.1007/978-3-030-39306-9_6
 16. Crespi, N., Drobot, A.T., Minerva, R.: The digital twin: What and why? In: Crespi, N., Drobot, A.T., Minerva, R. (eds.) *The Digital Twin*. pp. 3–20. Springer International Publishing, Cham (2023). https://doi.org/10.1007/978-3-031-21343-4_1, https://doi.org/10.1007/978-3-031-21343-4_1

17. Dalibor, M., Heithoff, M., Michael, J., Netz, L., Pfeiffer, J., Rumpe, B., Varga, S., Wortmann, A.: Generating customized low-code development platforms for digital twins. *Journal of Computer Languages* **70**, 101117 (Jun 2022). <https://doi.org/10.1016/j.cola.2022.101117>
18. Dalibor, M., Jansen, N., Rumpe, B., Schmalzing, D., Wachtmeister, L., Wimmer, M., Wortmann, A.: A cross-domain systematic mapping study on software engineering for digital twins. *Journal of Systems and Software* **193**, 111361 (2022). <https://doi.org/https://doi.org/10.1016/j.jss.2022.111361>, <https://www.sciencedirect.com/science/article/pii/S0164121222000917>
19. Dobaj, J., Riel, A., Krug, T., Seidl, M., Macher, G., Egretzberger, M.: Towards digital twin-enabled devops for cps providing architecture-based service adaptation & verification at runtime. In: *Proceedings of the 17th Symposium on Software Engineering for Adaptive and Self-Managing Systems*. p. 132–143. SEAMS '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3524844.3528057>, <https://doi.org/10.1145/3524844.3528057>
20. Dobaj, J., Riel, A., Macher, G., Egretzberger, M.: Towards DevOps for Cyber-Physical Systems (CPSs): Resilient Self-Adaptive Software for Sustainable Human-Centric Smart CPS Facilitated by Digital Twins. *Machines* **11**(10), 973 (Oct 2023). <https://doi.org/10.3390/machines11100973>, number: 10 Publisher: Multidisciplinary Digital Publishing Institute
21. Eramo, R., Bordeleau, F., Combemale, B., Brand, M.v.d., Wimmer, M., Wortmann, A.: Conceptualizing digital twins. *IEEE Software* **39**(2), 39–46 (2022). <https://doi.org/10.1109/MS.2021.3130755>
22. Garg, I., Hoffmann, D., Lüder, A., Rudolph, M.: Integrating domain expertise into dynamic digital models: A methodology for behaviour-driven production modelling. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. pp. 1–8 (2024). <https://doi.org/10.1109/ETFA61755.2024.10710744>
23. GitLab: What is a CI/CD pipeline? (2023), <https://about.gitlab.com/topics/ci-cd/cicd-pipeline/>
24. Gorla, A., Pezzè, M., Wuttke, J., Mariani, L., Pastore, F.: Achieving cost-effective software reliability through self-healing. *Computing and Informatics* **29**, 93–115 (01 2010)
25. Gregory, C., Mbolamananamalala, R., Rabah, S., Chapurlat, V.: Model based systems engineering applied to digital twin engineering: why and how to? *IFAC-PapersOnLine* **58**(19), 157–162 (2024). <https://doi.org/https://doi.org/10.1016/j.ifacol.2024.09.126>, <https://www.sciencedirect.com/science/article/pii/S2405896324015362>, 18th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2024
26. Gregory, C., Rabah, S., Chapurlat, V.: Exploring the digital thread: A comprehensive state of the art review. In: *IEEE International Conference on Digital Twin (DTwin 2005)*. IEEE, Calgary, Canada (Aug 2025). <https://doi.org/10.1109/SWC65939.2025.00266>, hybrid conference, August 18–22, 2025
27. Grieves, M.W.: Digital twins: Past, present, and future. In: Crespi, N., Drobot, A.T., Minerva, R. (eds.) *The Digital Twin*. pp. 97–121. Springer International Publishing, Cham (2023). https://doi.org/10.1007/978-3-031-21343-4_4, https://doi.org/10.1007/978-3-031-21343-4_4
28. Hasselbring, W., Henning, S., Latte, B., Möbius, A., Richter, T., Schalk, S., Wojcieszak, M.: Industrial devops. *CoRR abs/1907.01875* (2019). <https://doi.org/10.48550/arXiv.1907.01875>, <http://arxiv.org/abs/1907.01875>

29. Hugues, J., Hristosov, A., Hudak, J.J., Yankel, J.: Twinops - devops meets model-based engineering and digital twins for the engineering of cps. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings. MODELS '20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3417990.3421446>, <https://doi.org/10.1145/3417990.3421446>
30. Hugues, J., Yankel, J., Hudak, J., Hristozov, A.: TwinOps: Digital twins meets DevOps. Tech. rep., Carnegie-Mellon Univ. (2022)
31. Humble, J., Farley, D.: Continuous delivery: reliable software releases through build, test, and deployment automation. Pearson Education (2010)
32. ISO 23247-1:2021: Automation systems and integration — digital twin framework for manufacturing – part 1: Overview and general principles. Standard, International Organization for Standardization, Geneva, Switzerland (Oct 2021), 1st ed., 2021
33. ISO 30173:2023: Digital twin — Concepts and terminology. Standard, International Organization for Standardization, Geneva, Switzerland (Nov 2023), 1st ed., 2023
34. Jacoby, M., Usländer, T.: Digital twin and internet of things—current standards landscape. Applied Sciences **10**(18) (2020). <https://doi.org/10.3390/app10186519>, <https://www.mdpi.com/2076-3417/10/18/6519>
35. Kimmel, R., Michael, J., Wortmann, A., Zhang, J.: Digital twins for software engineering processes. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER). pp. 16–20 (2025). <https://doi.org/10.1109/ICSE-NIER66352.2025.00009>
36. Koren, I., Rinker, F., Meixner, K., Kröger, M., Zeng, M.: Implementing DevOps Practices in CPPS Using Microservices and GitOps. In: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA). pp. 1–4 (Sep 2023). <https://doi.org/10.1109/ETFA54631.2023.10275433>, iSSN: 1946-0759
37. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine learning operations (mlops): Overview, definition, and architecture. IEEE Access **11**, 31866–31879 (2023). <https://doi.org/10.1109/ACCESS.2023.3262138>
38. Lehman, M.M.: Laws of software evolution revisited. In: Montangero, C. (ed.) Software Process Technology. pp. 108—124. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)
39. Lehner, D., Wolny, S., Mazak-Huemer, A., Wimmer, M.: Towards a Reference Architecture for Leveraging Model Repositories for Digital Twins. In: 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). vol. 1, pp. 1077–1080 (Sep 2020). <https://doi.org/10.1109/ETFA46521.2020.9212109>, iSSN: 1946-0759
40. Lehner, D., Zhang, J., Pfeiffer, J., Sint, S., Splettstößer, A.K., Wimmer, M., Wortmann, A.: Model-driven engineering for digital twins: a systematic mapping study. Software and Systems Modeling **24**(5), 1339–1377 (2025). <https://doi.org/10.1007/s10270-025-01264-7>, <https://doi.org/10.1007/s10270-025-01264-7>
41. Marchand, J., Laval, J., Sekhari, A., Cheutet, V., Danielou, J.B.: Adaptive learning mode comparison for predictive maintenance systems. In: 11th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2025. vol. 59, pp. 112–117 (2025). <https://doi.org/https://doi.org/10.1016/j.ifacol.2025.09.021>, <https://www.sciencedirect.com/science/article/pii/S2405896325007827>

42. Marchand, J., Laval, J., Sekhari, A., Cheutet, V., Danielou, J.B.: End-to-end lifecycle machine learning framework for predictive maintenance of critical equipment. *Enterprise Information Systems* **19**(1-2), 2448008 (2025). <https://doi.org/10.1080/17517575.2024.2448008>, <https://doi.org/10.1080/17517575.2024.2448008>
43. Maree, J.M.J., Kruger, K., Basson, A.H.: Opportunities for Digital Twins for the provisioning, management and monitoring of heterogeneous IoT devices. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. pp. 378–389. *MODELS Companion '24*, Association for Computing Machinery, Linz, Austria (2024). <https://doi.org/10.1145/3652620.3688251>
44. Margaria, T., Pesch, D., McGibney, A.: Digital thread in smart manufacturing. In: Margaria, T., Steffen, B. (eds.) *Leveraging Applications of Formal Methods, Verification and Validation*. Practice. pp. 179–183. Springer Nature Switzerland, Cham (2022)
45. Mbolamananamalala, R., Rabah, S., Chapurlat, V.: Toward a model- and pattern-based method for the engineering and the maintenance of digital twin systems. In: *IEEE Digital Twin 2025 - The 2025 IEEE International Conference on Digital Twin*. pp. 1545–1553. IEEE, Calgary, Canada (Aug 2025). <https://doi.org/10.1109/SWC65939.2025.00244>, <https://imt-mines-ales.hal.science/hal-05227364>
46. Mejía Miranda, J., Muñoz, M., Rocha, A., Hernández Pérez, Y., Avila-George, H. (eds.): *New Perspectives in Software Engineering*. No. 1135 in *Studies in Computational Intelligence*, Springer Nature, 1st edn. (2024). <https://doi.org/10.1007/978-3-031-50590-4>
47. Miao, R., Liu, S., Sun, Y., Du, M., Bao, J.: Streamlining digital twin development and operation with dtops. *Journal of Intelligent Manufacturing* (2025). <https://doi.org/10.1007/s10845-025-02608-2>, <https://doi.org/10.1007/s10845-025-02608-2>
48. Michael, J., Cleophas, L., Zschaler, S., Clark, T., Combemale, B., Godfrey, T., Khelladi, D.E.e.a.: Model-driven engineering for digital twins: Opportunities and challenges. *Systems Engineering* **28**(5), 659–670 (2025). <https://doi.org/10.1002/sys.21815>, <https://incose.onlinelibrary.wiley.com/doi/abs/10.1002/sys.21815>
49. Microsoft Azure: What is DevOps? (2023), <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-devops>
50. Munappy, A.R., Mattos, D.I., Bosch, J., Olsson, H.H., Dakkak, A.: From ad-hoc data analytics to dataops. In: *Proceedings of the International Conference on Software and System Processes*. p. 165–174. *ICSSP '20*, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3379177.3388909>, <https://doi.org/10.1145/3379177.3388909>
51. Negrin, D.A.M., Cleophas, L., Van Den Brand, M.: Using Ptolemy II as a Framework for Virtual Entity Integration and Orchestration in Digital Twins. In: *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. pp. 233–236 (Oct 2021). <https://doi.org/10.1109/MODELS-C53483.2021.00041>
52. Pfeiffer, J., Lehner, D., Wortmann, A., Wimmer, M.: Modeling Capabilities of Digital Twin Platforms - Old Wine in New Bottles? *The Journal of Object Technology* **21**(3), 3:1 (2022). <https://doi.org/10.5381/jot.2022.21.3.a10>
53. Red Hat: What is a CI/CD pipeline? (2024), <https://www.redhat.com/en/topics/devops/what-cicd-pipeline>

54. Romero-Garcés, A., Hidalgo-Paniagua, A., González-García, M., Bandera, A.: On managing knowledge for mape-k loops in self-adaptive robotics using a graph-based runtime model. *Applied Sciences* **12**(17) (2022). <https://doi.org/10.3390/app12178583>, <https://www.mdpi.com/2076-3417/12/17/8583>
55. Singh, M., Fuenmayor, E., Hinchy, E.P., Qiao, Y., Murray, N., Devine, D.: Digital twin: Origin to future. *Applied System Innovation* **4**(2) (2021). <https://doi.org/10.3390/asi4020036>, <https://www.mdpi.com/2571-5577/4/2/36>
56. Talasila, P., Gomes, C., Mikkelsen, P.H., Arboleda, S.G., Kamburjan, E., Larsen, P.G.: Digital twin as a service (dtaas): A platform for digital twin developers and users. In: 2023 IEEE Smart World Congress (SWC). pp. 1–8 (2023). <https://doi.org/10.1109/SWC57546.2023.10448890>
57. Tamburri, D.A.: Sustainable mlops: Trends and challenges. In: 2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC). pp. 17–23 (2020). <https://doi.org/10.1109/SYNASC51798.2020.00015>
58. Tan, B., Matta, A.: The digital twin synchronization problem: Framework, formulations, and analysis. *IISE Transactions* **56**(6), 652–665 (2024). <https://doi.org/10.1080/24725854.2023.2253869>, <https://doi.org/10.1080/24725854.2023.2253869>
59. Unity: What is CI/CD? (2025), <https://unity.com/topics/what-is-ci-cd>
60. Waseem, M., Liang, P., Shahin, M.: A systematic mapping study on microservices architecture in devops. *Journal of Systems and Software* **170**, 110798 (2020). <https://doi.org/https://doi.org/10.1016/j.jss.2020.110798>, <https://www.sciencedirect.com/science/article/pii/S0164121220302053>
61. Weaveworks: GitOps — Operations by Pull Request (Sep 2018), <https://medium.com/weaveworks/gitops-operations-by-pull-request-14e8b659b058>
62. Zeamari, D., Gregory, C., Rabah, S., Chapurlat, V.: Digital twin engineering guided by mbsse and digital thread. In: Proceedings of the 2025 IEEE International Conference on Digital Twin (DTwin). IEEE, Calgary, Canada (Aug 2025). <https://doi.org/10.1109/SWC65939.2025.00249>, <https://imt-mines-ales.hal.science/hal-05253906>